PRACTICAL NO. 1

AIM:

● Solving 2 water jug Problem: Write a program to solve the 2-water jug problem by manually providing moves from the user.

● Write a program to implement Missionaries and Cannibals problem. Constraints: three missionaries and three cannibals must cross a river using a boat which can carry at most two people, under the constraint that, for both banks, if there are missionaries present on the bank, they cannot be outnumbered by cannibals (if they were, the cannibals would eat the missionaries). The boat cannot cross the river by itself with no people on board.

THEORY:

2 Jug water problem and missionaries and cannibals' problem is best example of mixture of logic and concept.

In 2 water jug problem 2 jugs of fixed capacity are provided (say 3 and 5) we have a desired amount of water as our goal (4 lit) the code is like a game if the user enters correct path they win if either of the jug have desired quantity of water by pouring emptying or filling the jugs (amount of water is unlimited)

In missionaries and cannibals' there are 3 of each of them, a boat is used to carry them from one shore of river to another with constraint that if the cannibals' extend the missionary they eat them therefore its not valid also boat can either carry one or two of them at a time. We use bfs for searching all the possible paths and select the valid one and proceed till the goal is achieved.

ALGORITHM/METHODOLOGY:

Two Water Jug Problem:

- Read the capacity of jug X (capx) and jug Y (capy).
- Read the goal state as a pair (goal_x, goal_y).
- Initialize the current amount of water in both jugs: x = 0, y = 0.
- Repeat the following steps until (x, y) becomes equal to the goal state:

  Display menu of operations to the user.

  Read the chosen operation.

- Perform the selected operation:

1. Fill jug X completely.
2. Fill jug Y completely.

3. Empty jug X.
4. Empty jug Y.
5. Pour water from jug X to jug Y until X is empty or Y is full.
6. Pour water from jug Y to jug X until Y is empty or X is full.

- After each operation, display the current state (x, y).
- If the user chooses any invalid option, exit the loop.
- When the goal state is reached, display "Goal reached".

Missionaries and cannibals' problem

- Read the start state (missionaries, cannibals, boat) and the goal state.
  Boat= 0 leftside
  Boat = 1 right side
- Initialize a queue for BFS with:

  the start state

  path containing only the start state

- Initialize an empty visited set.
- While the queue is not empty:

1. Remove the first element from the queue.
2. Mark the current state as visited.

- If the current state is the goal state return the path (solution found).

Generate all possible next states using valid boat moves:

- (1 missionary)
- (1 cannibal)
- (1 missionary, 1 cannibal)
- (2 missionaries)
- (2 cannibals)
- Move people depending on boat position.

Check each generated state using Permissible():

- Missionaries and cannibals must be between 0 and 3
- Cannibals must never outnumber missionaries on either side

If the next state is safe and not visited,add it to the queue with the updated path.

If the queue becomes empty and the goal is not found,return None (no solution)

Display the solution path and stop the program.

DATASET DESCRIPTION:

Each data instance represents the amount of water in two jugs and is defined as:

1. x: Amount of water in jug X (0 to capacity of X)
2. y: Amount of water in jug Y (0 to capacity of Y)

So, each state is represented as: (x, y).

Each data instance is represented as a tuple in cannibals and missionaries problem:

1. Missionaries (mis): Number of missionaries on the left bank (0–3)
2. Cannibals (can): Number of cannibals on the left bank (0–3)
3. Boat (boat): Boat position
    - 0 = Left bank
    - 1 = Right bank

So, each state is of the form: (mis, can, boat).

Size:

- Maximum possible states: 32

IMPLEMENTATION:

Two Water jug problem:

```
def two_water_jugs (capx , capy, goal):
  x,y = [0,0]
  while ( x != goal[0] or y != goal[1]):
    opt= int(input("choose your operation "))
    if(opt == 1):
      x=capx
      print("x is filled")
    elif(opt == 2):
      y=capy
```

```
      print("y is filled")
    elif(opt == 3):
      x=0
      print("x is emptied")
    elif(opt == 4):
      y=0
      print("y is emptied")
    elif(opt == 5):
      space =capy-y
      pour= min(x,space)
      x= x-pour
      y= y+pour
      print("x is poured in y")
    elif(opt == 6):
      space =capx-x
      pour= min(y,space)
      x= x+pour
      y= y-pour

      print("y is poured in x")
    else:
      print("exit")
      w=1
      break
    print(x,y)
  print("goal reached")


print("enter capacity of x,y and targeted value")
x= int(input("capacity of x "))
y= int(input("capacity of y "))
```

```
print("enter targeted value")
z= list(map(int,input().split()))
two_water_jugs(x,y,z)
```

Missionaries and Cannibals problem:

```
def bfs(start, goal):
  queue=[(start,[start])]
  visited=set()
  while queue:
    state,path=queue.pop(0)
    visited.add(state)
    if state==goal:
      return path
    for next_state in nextstate_gene(state):
      if next_state not in visited:
        queue.append((next_state,path+[next_state]))
  return None

def nextstate_gene(state):
  nextstate=[]
  mis,can,boat=state
  move=[(1,0),(0,1),(1,1),(2,0),(0,2)]
  for m,c in move:
    if m + c == 0 or m + c > 2:
      continue
    if boat==0:
      ns=(mis-m,can-c,1)
    else:
      ns=(mis+m,can+c,0)
```

```
    if Permissible(ns):
      nextstate.append(ns)
  return nextstate

def Permissible(state):
  mis,can,boat=state
  if mis<0 or mis>3 or can<0 or can>3:
    return False
  if 3-mis>0 and 3-mis<3-can:
    return False
  if mis>0 and mis<can:
    return False
  return True


print("Enter start state ")
s_m, s_c, s_b = map(int, input().split())

print("Enter goal state ")
g_m, g_c, g_b = map(int, input().split())

start = (s_m, s_c, s_b)
goal = (g_m, g_c, g_b)

print(bfs(start, goal))
```

OUTPUT:

```
enter capacity of x,y and targeted value
capacity of x 3
capacity of y 5
enter targeted value
0 4
choose your operation 2
y is filled
0 5
choose your operation 6
y is poured in x
3 2
choose your operation 3
x is emptied
0 2
choose your operation 6
y is poured in x
2 0
choose your operation 2
y is filled
2 5
choose your operation 6
y is poured in x
3 4
choose your operation 3
x is emptied
0 4
goal reached
```

```
Enter start state
3 3 0
Enter goal state
0 0 1
[(3, 3, 0), (2, 2, 1), (3, 2, 0), (3, 0, 1), (3, 1, 0), (1, 1, 1), (2, 2, 0), (0, 2, 1), (0, 3, 0), (0, 1, 1), (1, 1, 0), (0, 0, 1)]
```

RESULT ANALYSIS

1. Water Jug Problem

The Water Jug algorithm effectively simulates every plausible operation such as filling, emptying, and pouring water between the two containers. Starting from the initial state (0,0), it proceeds to generate the states through the choice of action.

The state variable tracks the current state (x, y) correctly.It terminates successfully when the target state is reached.This program makes sure the water level is never more than the jug's capacity or becomes negative.From this, it can be derived that the target posed can be achieved if, and only if, it is mathematically attainable from the presented capacity of each jug.

2. Missionaries and Cannibals Problem - BFS

The particular algorithm systematically searches all attainable states that are accessible from the initial state.BFS will ensure that the shortest and safest way from start to goal is obtained.

Invalid states, which have more cannibals than missionaries, are not possible and are removed with the use ofThe use of the queue and the set of visited eliminates the infinite loop. The algorithm discovers a correct solution path upon a valid existing solution.

QUESTIONS:

1.Why is BFS preferred over DFS here?

BFS is preferred because it moves from level to level checking each valid node that can be used giving the shortest path.

2.What happens if boat capacity is increased by 3?

The problem then becomes less complex. But only 2 missionaries and a cannibal or either all missionary or all cannibals will be allowed it its 3

3.Can the water jug problem be solved instead of manual input? How?

Yes, it can be solved using search teqniques

4.How do constraints prune invalid state?

In Water Jug: prevents negative or overflow amounts. In Missionaries & Cannibals: prevents missionaries from being outnumbered, so only safe states are explored.

CONCLUSION:

Both the Two Water Jug problem and the Missionaries and Cannibals problem describe, in general terms, the concept of Artificial Intelligence and its ability to solve complex problems. Moreover, the Water Jug problem explains the interactive problems and the role that user-defined operations play in the problem-solving approach, whereas the Mission and Cannibal problem explains the concept of Breadth First Search and its ability to find the optimum solution without violating the constraints, thus showing the relevance and role played in the real-world problem-solving approach in terms of the transition between states and the valid moves available to solve complex problems.